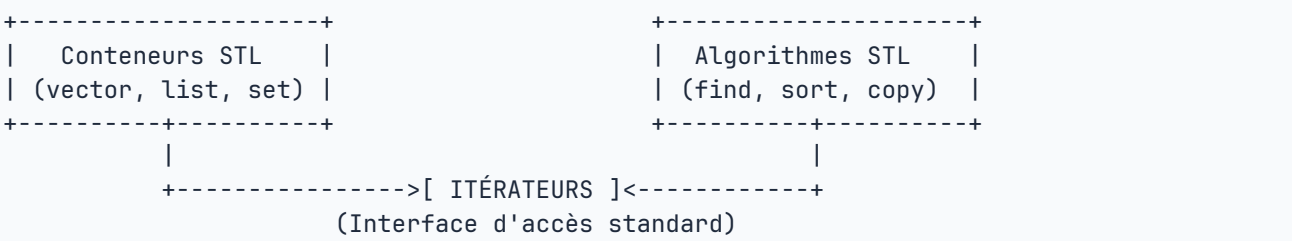
std::vector & les Itérateurs en C++

Architecture interne, Design Pattern Iterator, mémoire et cas pratiques

1. Philosophie : Pourquoi les Itérateurs ? (Design Pattern Iterator)

Dans la bibliothèque standard (STL) conçue par **Alexander Stepanov**, l'objectif fondamental était de résoudre le problème d'explosion combinatoire entre *conteneurs* et *algorithmes* :

- Si vous avez **N conteneurs** (vector, list, deque, set...) et **M algorithmes** (find, sort, count, copy...), sans abstraction, il faudrait écrire `N × M` fonctions distinctes.
- **La solution de la STL :** Insérer un pont universel : **l'itérateur**. Les algorithmes ne manipulent jamais directement le conteneur, ils manipulent uniquement une paire d'itérateurs `[first, last)`.



> 💡 **Définition Formelle**
>
> Un itérateur est une généralisation du concept de **pointeur**. C'est un objet (ou un pointeur brut) qui permet de parcourir les éléments d'une séquence et d'accéder à leur valeur via la surcharge des opérateurs : `*it`, `it++`, `it != end`, `it->member`.

2. Anatomie Interne de `std::vector`

Contrairement aux tableaux statiques C (`int tab[10]`) ou aux listes chaînées (`std::list`), `std::vector` est un **tableau dynamique contigu en mémoire**.

Structure en mémoire (Stack vs Heap)

Sur la pile (Stack), un objet `std::vector` n'occupe que **24 octets** (sur architecture 64 bits), constitués de 3 pointeurs bruts :

- `_M_start` : pointe sur le 1er élément alloué sur le tas.
- `_M_finish` : pointe juste après le dernier élément valide (correspond à `end()`).
- `_M_end_of_storage` : pointe sur la fin du bloc mémoire actuellement réservé.



Différence Cruciale : `size()` vs `capacity()`

Propriété	Méthode	Définition
Taille (Size)	`vec.size()`	Nombre d'éléments réels actuellement construits (`_M_finish - _M_start`).
Capacité (Capacity)	`vec.capacity()`	Nombre total d'éléments que le vector peut contenir avant d'avoir à réallouer la mémoire (`_M_end_of_storage - _M_start`).

La Réallocation Dynamique & Coût Amorti Γ(1)

Quand `size() == capacity()` et qu'on effectue un `push_back()` :

1. Le vector alloue un **nouveau bloc** plus grand ailleurs sur le tas (généralement le double : facteur 2 en GCC/Clang, 1.5 en MSVC).
2. Il **déplace (move)** ou copie les anciens éléments vers la nouvelle zone mémoire.
3. Il détruit les anciens éléments et libère l'ancienne mémoire.
4. Il ajoute le nouvel élément.

> ⚠ **Danger : Invalidation des Itérateurs !**
>
> Lors d'une réallocation, l'ancien bloc mémoire est désalloué ! Tout pointeur, référence ou itérateur pointant sur l'ancien bloc devient un **dangling pointer** (pointeur sauvage). L'accès à cet itérateur provoque un *Undefined Behavior* (crash ou corruption mémoire).
>
> **Bonne pratique :** Si vous connaissez à l'avance le nombre d'éléments, utilisez `vec.reserve(N)` pour éviter les réallocations inutiles.

3. La Hiérarchie des Catégories d'Itérateurs

Tous les itérateurs ne permettent pas les mêmes opérations. Le C++ définit des catégories précises :

Catégorie	Opérations permises	Exemple de conteneur / source
Input	Lecture seule, 1 seul passage séquentiel (`*it` , `++it`)	`std::istream_iterator`
Output	Écriture seule, 1 seul passage (`*it = val` , `++it`)	`std::ostream_iterator` , `std::back_inserter`
Forward	Lecture/Écriture, passages multiples vers l'avant	`std::forward_list` (liste simplement chaînée)
Bidirectional	Vers l'avant et vers l'arrière (`++it` , `--it`)	`std::list` , `std::set` , `std::map`
Random Access	Saut direct en temps constant Γ(1) (`it + n` , `it[n]` , `it1 < it2`)	`std::deque`
Contiguous	Random Access + éléments physiquement adjacents en mémoire	`std::vector` , `std::array` , `std::string` , pointeur brut `T*`

4. Les 4 Types d'Itérateurs d'un Vector

Pour un conteneur `std::vector<T>` , il existe 4 types de parcours :

- `vector<T>::iterator` : Début avec `begin()`, fin avec `end()` (modifiable).
- `vector<T>::const_iterator` : Début avec `cbegin()`, fin avec `cend()` (lecture seule).
- `vector<T>::reverse_iterator` : Début avec `rbegin()`, fin avec `rend()` (parcours inversé).
- `vector<T>::const_reverse_iterator` : Début avec `crbegin()`, fin avec `crend()` (parcours inversé lecture seule).

> 🔑 **Convention Fondamentale STL : L'intervalle semi-ouvert [begin, end)**
> `end()` et `rend()` ne pointent **JAMAIS** sur un élément valide ! Ils pointent sur la position fictive « *un cran après le dernier élément* ». C'est ce qui permet d'écrire la condition d'arrêt : `it != vec.end()`.

5. Analyse Détaillée de votre Code (L'Énigme du Reverse Iterator)

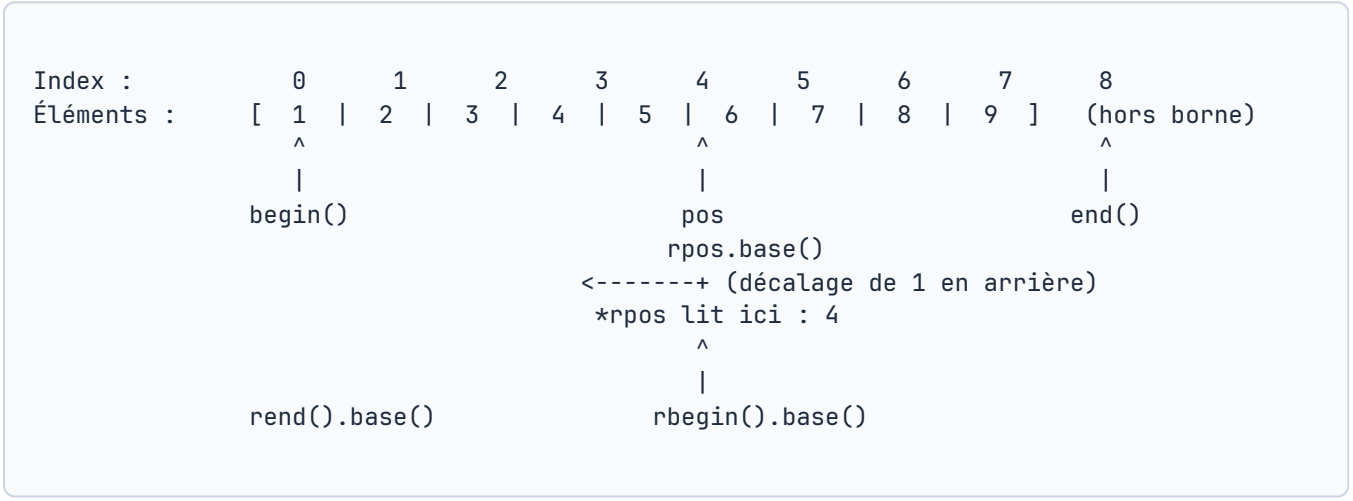
Dans votre code `example1.cpp`, vous avez observé :

```
vector<int> coll = {1, 2, 3, 4, 5, 6, 7, 8, 9};
auto pos = find(coll.cbegin(), coll.cend(), 5); // pos pointe sur 5
vector<int>::const_reverse_iterator rpos(pos);

cout << "*pos: " << *pos << endl;   // Affiche 5
cout << "*rpos: " << *rpos << endl; // Affiche 4 ! Pourquoi ?!
```

Pourquoi `*rpos` donne-t-il 4 ?

Pour comprendre ce comportement, il faut observer comment `rbegin()` et `rend()` sont définis par rapport à `begin()` et `end()` :



La Relation Mathématique STL :

Pour qu'un `reverse_iterator` puisse parcourir toute la collection de droite à gauche en respectant la convention semi-ouverte :

- `rbegin()` est construit à partir de `end()`.
- `rend()` est construit à partir de `begin()`.
- Quand on déréférence `*rpos`, la STL exécute : `*(rpos.base() - 1)`.

Puisque `rpos` a été initialisé avec `pos` (qui pointe sur 5), son `base()` est `pos`. Donc déréférencer `*rpos` renvoie l'élément situé juste **avant** `pos`, c'est-à-dire **4** !

6. Pièges Fréquents & Bonnes Pratiques

Piège n°1 : Invalidation d'itérateur lors d'une suppression

```cpp
// ERREUR GRAVE : it est invalidé dès que erase() est appelé !
for (auto it = vec.begin(); it != vec.end(); ++it) {
    if (*it == 5) {
        vec.erase(it); // CRASH ou Undefined Behavior au prochain ++it !
    }
}

// LA BONNE MÉTHODE (C++98 / C++11) : erase() retourne le prochain itérateur valide
for (auto it = vec.begin(); it != vec.end(); ) {
    if (*it == 5) {
        it = vec.erase(it); // mise à jour propre de it
    } else {
        ++it;
    }
}

// LA MÉTHODE MODERNE (C++20) : std::erase
std::erase(vec, 5);
```

✔ Synthèse des Règles d'Or

1. Préférez `cbegin()` / `cend()` si vous n'avez pas besoin de modifier l'élément.
2. Préférez les boucles par intervalle (Range-based for) `for (const auto& x : vec)` quand vous n'avez pas besoin de l'indice ou de l'itérateur.
3. Utilisez `std::reserve()` avant d'insérer beaucoup d'éléments avec `push_back()`.
4. Ne stockez jamais un itérateur à travers des opérations qui modifient la taille du vecteur (`push_back`, `insert`, `erase`, `resize`).